6. 调试和错误处理

| 2023-03-04

This document contains Cognex Corporation confidential information, which must be protected and used only in accordance with the Mutual Confidentiality Agreement entered into by Cognex Corporation and Apple Inc. dated November 1, 2014

COGNEX

6.1 Visual Studio 中的调试

1. 非中断 (正常) 模式下的调试
   1) 输出调试信息： Debug.WriteLine(), Trace.WriteLine() 。这两种方法需要使用命名空间 **System.Diagnostice**
   2) 跟踪点 ： 一种把信息输出到 Output 窗口中的办法是使用跟踪点 (tracepoint) 。与 Debug.WriteLine() 相同。
2. 中断模式下的调试
   1) 进入中断模式
      a. 断点：是源代码中自动进入中断模式的一个标记。
      b. Debug.Assert()
      c. Trace.Assert()
   2) 监视变量的内容
   3) 单步执行代码：
      a. Step Into— 执行并移动到下一条要执行的语句上，快捷键 F11 。
      b. Step Over— 同上，但不进入嵌套的代码块，包括函数，快捷键 F10 。
      c. Step Oue— 执行到代码块的末尾处，在执行完该语句块后，重新进入中断模式，快捷键 Shift+F11 。
   4) Immediate 和 Command 窗口
   5) Call Stack 窗口：描述程序是如何执行到当前位置的。

© 2022 Cognex Corporation

6.2 错误处理

1. **Try…catch…finally**
   1) try—— 包含抛出异常的代码。
   2) catch—— 包含抛出异常时要执行的代码。
   3) finally—— 包含始终会执行的代码 ()
   4) 函数 **ThrowException()** ，这个函数会根据调用时使用的参数抛出异常： ThrowException(“none”)— 不抛出异常； ThrowException(“simple”)— 生成一般异常； ThrowException(“index”)— 生成 System.IndexOutOfRangeException 异常； ThrowException(“nested index”)— 包含它自己的 try 块，其中的代码调用 ThrowException(“index”) ，生成一个 System.IndexOutOfRangeException 异常。
2. 列出和配置异常：

   .Net Framework 包含许多异常类型，可以在代码中自由抛出和处理这些类型的异常。 IDE 提供了一个 Exception 窗口，可以检查和编辑可用的异常。
3. 异常处理的注意事项

   如果在当前 catch 块中抛出异常可以使用表达式：

   throw;

   这个表达式会再次抛出 catch 块处理过的异常。如果以这种方式抛出异常，该异常就不会由当前的 try…catch…finally 块处理，而是由上一级的代码处理 (但是嵌套结构中的 finally 块仍会执行) 。

© 2022 Cognex Corporation

6.3 练习

下面定义了一个枚举数据类型 orientation 。编写一个控制台程序，使用结构化异常处理 (SEH) 将 byte 类型的变量安全地强制转换为 orientation 类型变量。注意，可以使用 checked 关键字强制抛出异常，下面是一个示例。在你编写的应用程序中应该使用这段代码。

```
enum Orientation： byte
{
    North = 1，
    South  = 2，
    East = 3，
    West = 4
}
myDirection = checke((Orientation) myByte);
```


© 2022 Cognex Corporation

Thank you

© 2022 Cognex Corporation